

LAPORAN TUGAS 3
PRAKTIKUM TEKNOLOGI *CLOUD COMPUTING*



Disusun Oleh:

Afif Khaliq Ramadhan / 123230177

Program Studi Informatika

Jurusan Informatika

Fakultas Teknik Industri

Universitas Pembangunan Nasional “Veteran” Yogyakarta

2026

1. Penjelasan Singkat Aplikasi Tugas 2

Aplikasi ini adalah Aplikasi Notes (Catatan) yang berfungsi untuk mengelola catatan secara penuh (memiliki fitur CRUD: Create, Read, Update, Delete).

- Backend: Menyediakan API yang dibangun menggunakan Node.js, Express, dan terkoneksi ke database MySQL melalui ORM Sequelize.
- Frontend: Menyediakan UI interaktif yang memiliki fitur pencarian, statistik catatan, dan notifikasi responsif dengan desain.

2. Penjelasan Teknologi Frontend

Proyek frontend dibangun menggunakan teknologi dasar web, yaitu HTML, CSS, dan JavaScript murni tanpa menggunakan framework tambahan seperti React atau Vue. Meskipun tidak menggunakan framework, aplikasi ini berjalan di server statis menggunakan module Node.js yaitu serve pada scripts di package.json.

3. Penjelasan Skenario Deployment

Proyek ini menggunakan layanan dari Google Cloud Platform dengan otomatisasi dari Cloud Build.

A. Skenario Deployment Backend

Repository pada branch backend akan di-deploy menggunakan layanan kontainer serverless, Google Cloud Run. Skenario deployment ini dilakukan secara langsung melalui pengaturan di antarmuka Cloud Run. Layanan ini akan membaca file Dockerfile yang bertugas mendefinisikan image Node.js ringan, melakukan instalasi paket dengan npm install, dan menjalankan server di port 3000.

Langkah - langkah Deployment Backend ke Cloud Run

- Buka Cloud Run: Masuk ke Google Cloud Console dan buka layanan Cloud Run.
- Pilih Connect Repo: Buat service baru dan pilih opsi untuk menghubungkan (connect) langsung ke repository GitHub yang berisi kode backend.
- Setting Konfigurasi: Lakukan pengaturan konfigurasi dasar layanan, seperti memilih region dan memastikan opsi allow unauthenticated invocations dicentang agar API bisa diakses.
- Ubah Container Port: Pada bagian konfigurasi kontainer, sesuaikan Container Port menjadi 3000 (sesuai port server Node.js).
- Masukkan .env: Masukkan seluruh konfigurasi kredensial database (variabel dari file .env) ke dalam tab Environment Variables.
- Klik Tombol Create: Klik Create untuk menyimpan dan memicu proses build.
- Menunggu Hasil Deployment: Tunggu proses instalasi dan perakitan selesai hingga Cloud Run menerbitkan URL endpoint API yang siap digunakan.

B. Skenario Deployment Frontend

Repository pada branch frontend akan dimasukkan ke App Engine. Konfigurasi lingkungannya diatur dalam file app.yaml yang menentukan penggunaan runtime Node.js versi 20 dan mengatur skalabilitas otomatis (automatic scaling) untuk menghemat biaya dengan membatasi jumlah instance maksimal menjadi satu.

Berbeda dengan backend, proses deployment frontend ini murni diotomatisasi dengan membuka layanan Cloud Build dan mengeksekusi file cloudbuild.yaml. File ini menggunakan image Cloud SDK untuk mengatur batasan waktu build dan langsung menjalankan eksekusi perintah gcloud app deploy tanpa harus membuat image Docker terlebih dahulu.

Langkah - langkah Deployment Frontend ke App Engine via Cloud Build

- Buka Cloud Build: Masuk ke Google Cloud Console dan buka layanan Cloud Build.
- Create Trigger: Buka menu pemicu (Triggers) dan klik tombol Create Trigger.
- Isi Detail Trigger: Isi form pemicu, beri nama, dan hubungkan dengan repository GitHub yang berisi kode frontend.
- Configuration dengan cloudbuild.yaml: Pada bagian Configuration, arahkan eksekusi agar menggunakan file cloudbuild.yaml yang ada di repository.
- Pilih Service Account: Pilih Service Account yang tepat (yang memiliki izin khusus untuk mendeploy aplikasi App Engine).
- Klik Tombol Create: Klik Create untuk menyimpan trigger, lalu jalankan.
- Menunggu Hasil Deployment: Tunggu hingga Cloud Build selesai mengeksekusi instruksi di dalamnya dan URL aplikasi frontend berhasil diluncurkan ke layanan App Engine.

4. Perbandingan Singkat Berdasarkan Hasil Deployment

Cloud Run (Backend)	App Engine (Frontend)
Menggunakan arsitektur berbasis container (Docker) yang secara otomatis mendeteksi, merakit, dan menjalankan image dari repository.	Bertindak murni sebagai Platform as a Service (PaaS) menggunakan file app.yaml tanpa perlu merakit kontainer Docker baru
Sangat ideal untuk layanan API karena memberikan fleksibilitas tinggi pada konfigurasi environment server di dalam kontainer.	Sangat ideal dan praktis untuk menyajikan aplikasi antarmuka web statis tanpa memerlukan konfigurasi OS yang rumit.
Integrasi rahasia dan kredensial sangat mudah dikelola melalui pengisian Environment Variables di konsol.	Proses otomatisasi diikat menggunakan pemicu Cloud Build dan Service Account khusus demi menjamin keamanan peluncuran aplikasi.
Memiliki responsibilitas scale to zero yang sangat cepat sehingga tangguh dan efisien menghadapi lonjakan permintaan API.	Skalabilitas dikontrol secara ketat pada konfigurasinya untuk memastikan penghematan biaya.

5. Link URL Backend dan Frontend

frontend : <https://prak-cc-01.et.r.appspot.com/>

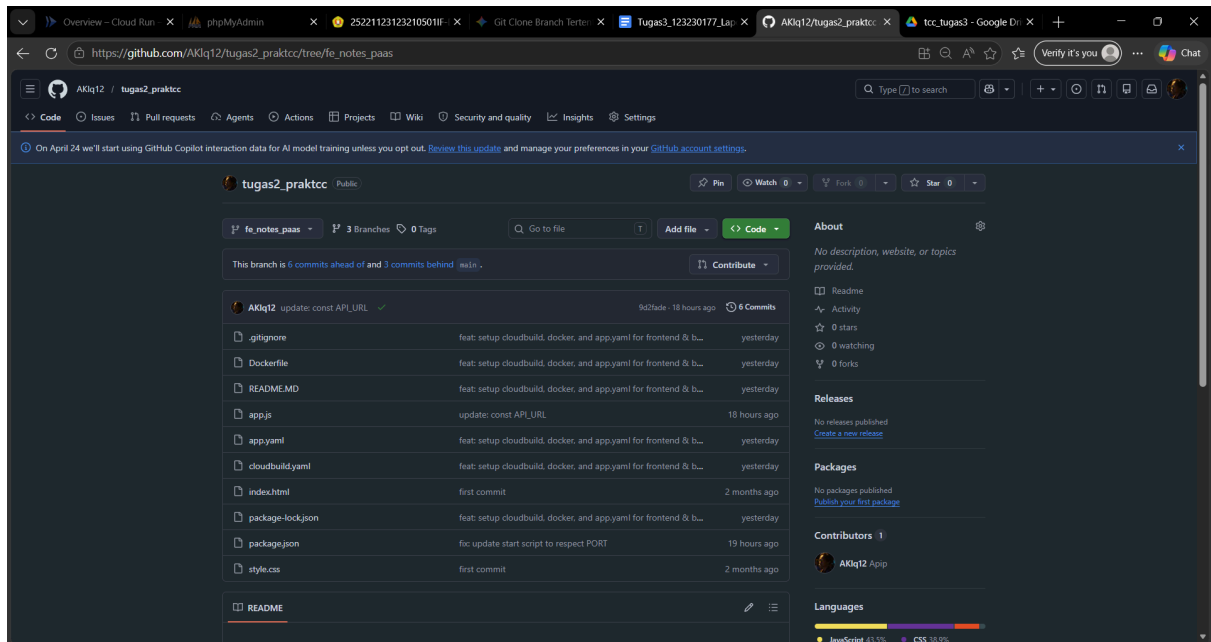
backend : <https://be-tugas3-tcc-177-385639935267.us-central1.run.app>

6. Link Repository

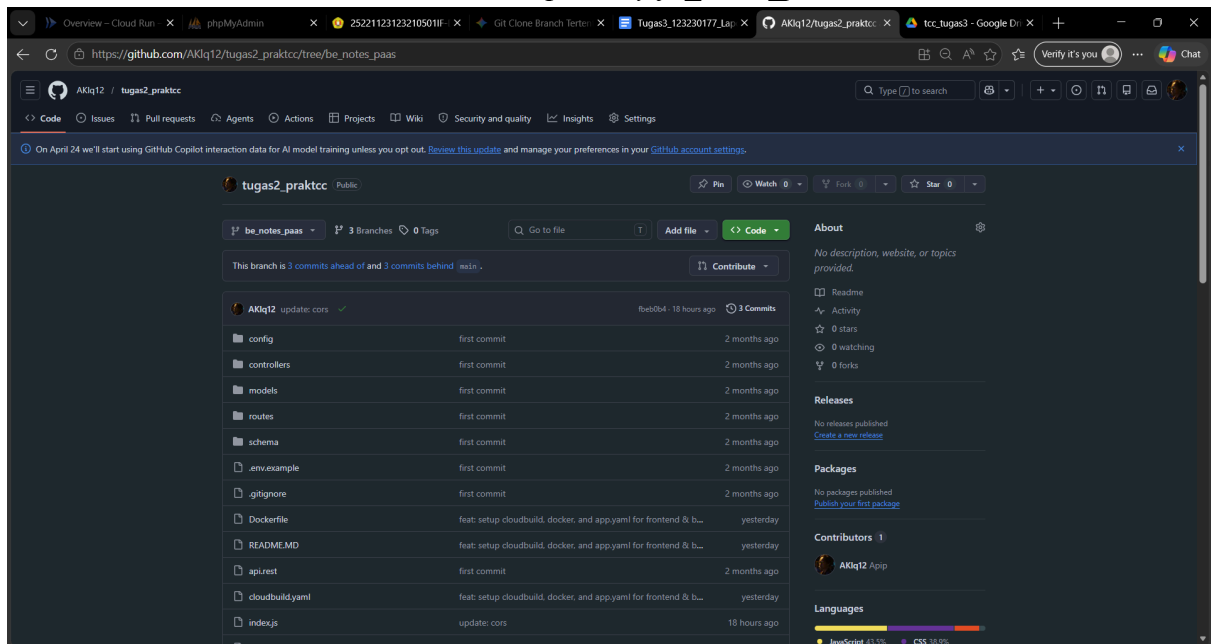
https://github.com/AKlq12/tugas2_praktcc/tree/fe_notes_paas

https://github.com/AKlq12/tugas2_praktcc/tree/be_notes_paas

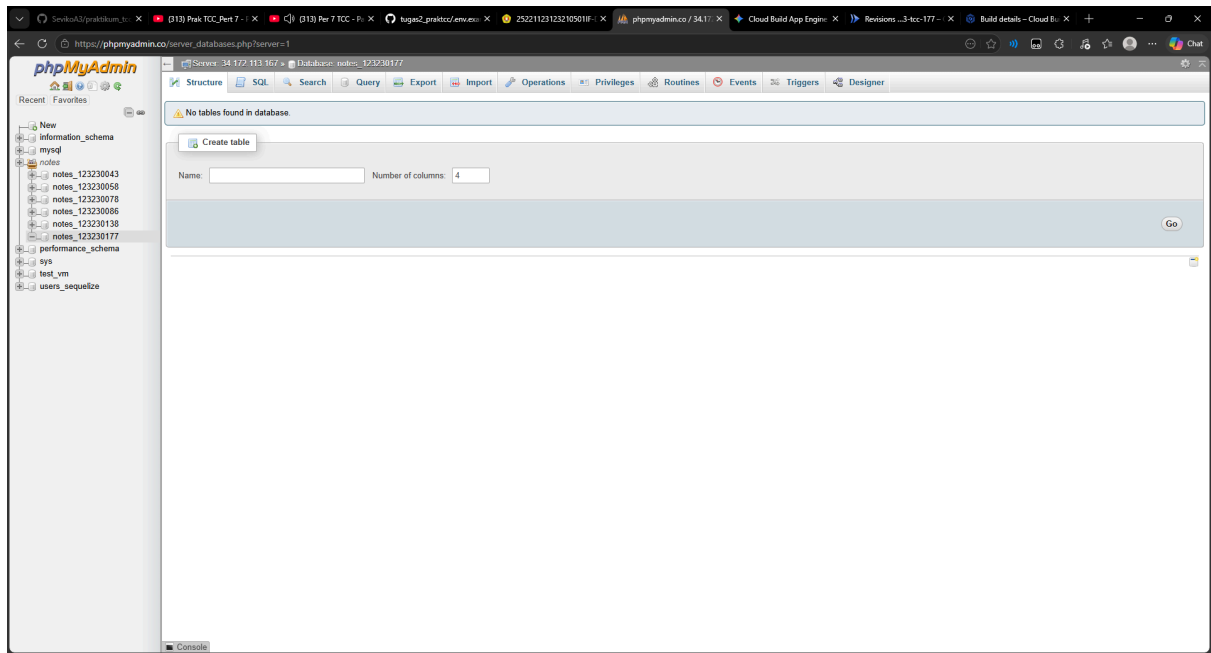
7. Hasil Screenshot



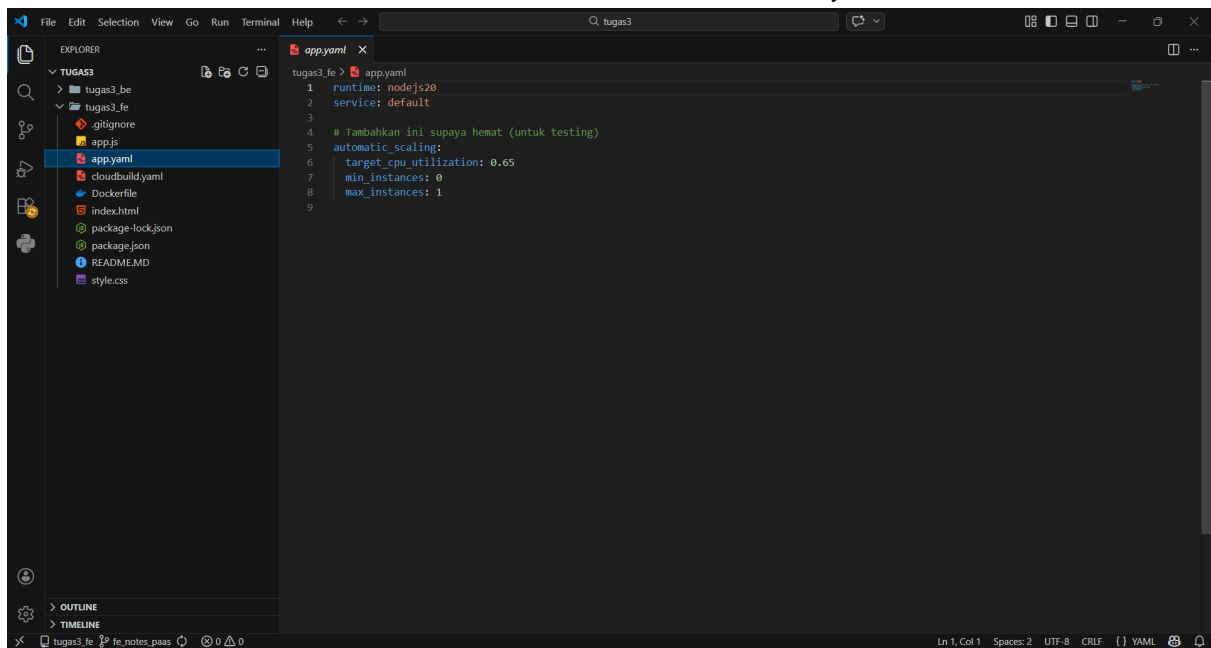
Gambar 5.1 Repository fe_notes_paas



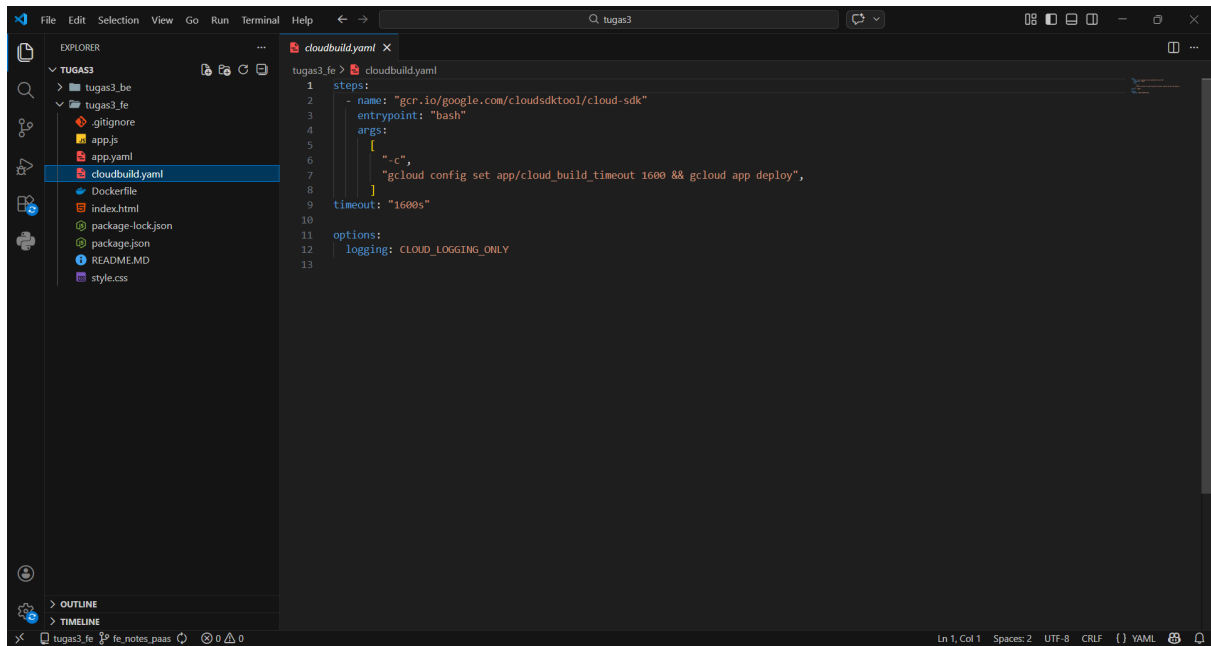
Gambar 5.2 Repository be_notes_paas



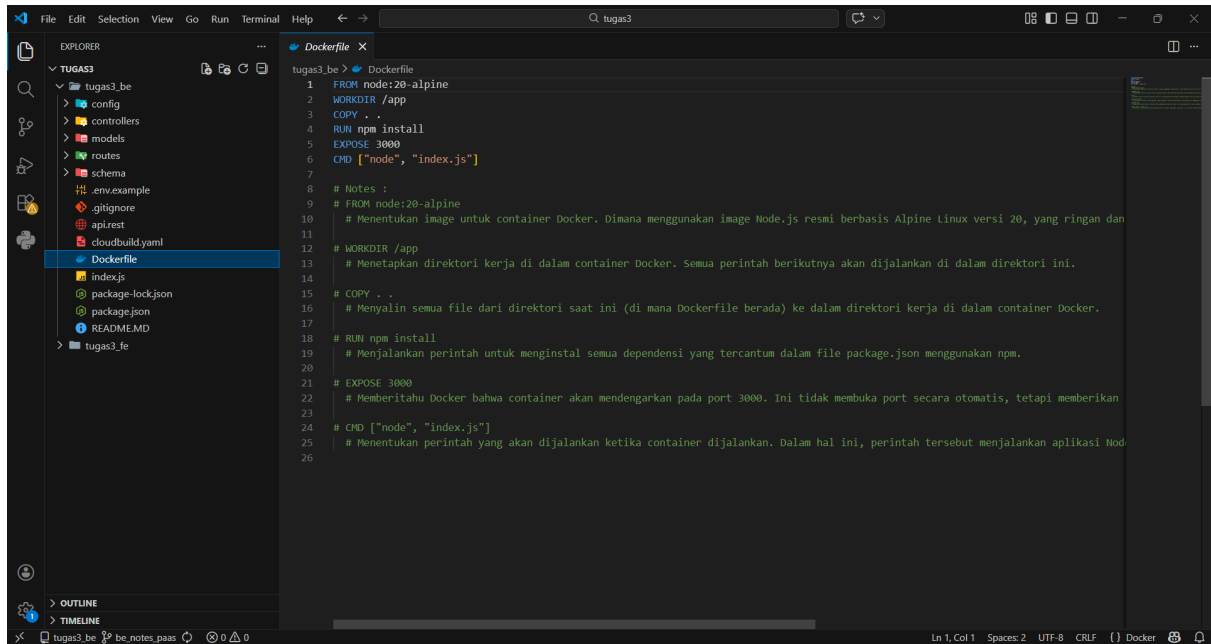
Gambar 5.3 Pembuatan Database di PHPMYAdmin



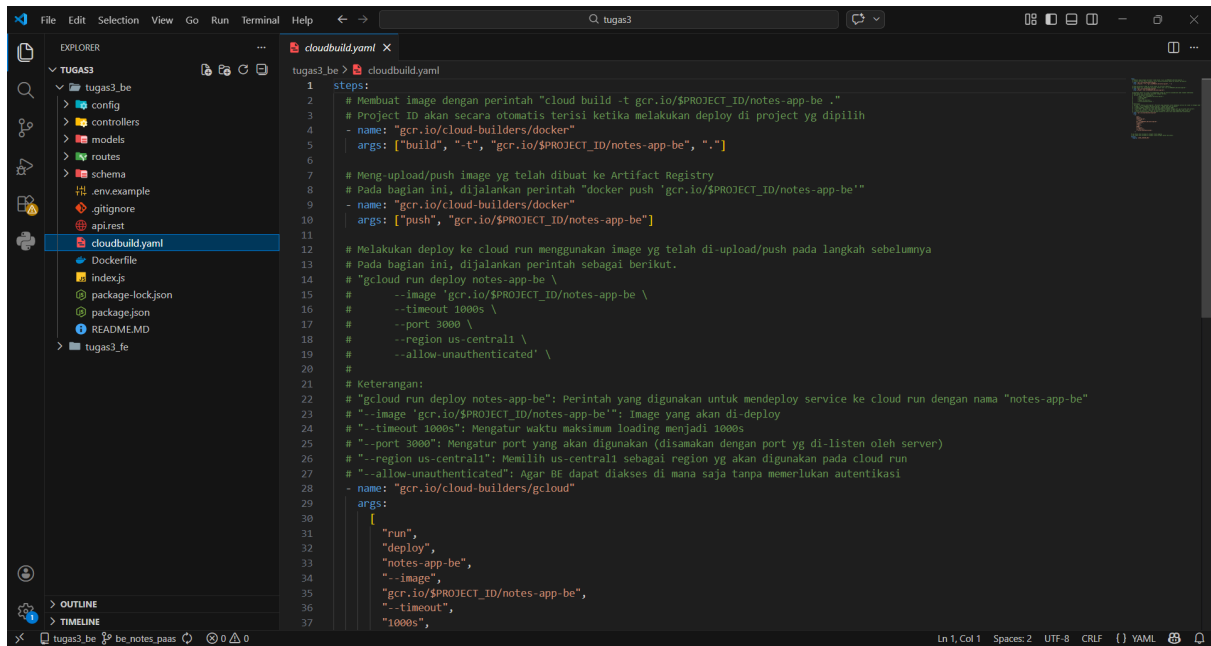
Gambar 5.4 File Konfigurasi app.yaml di frontend



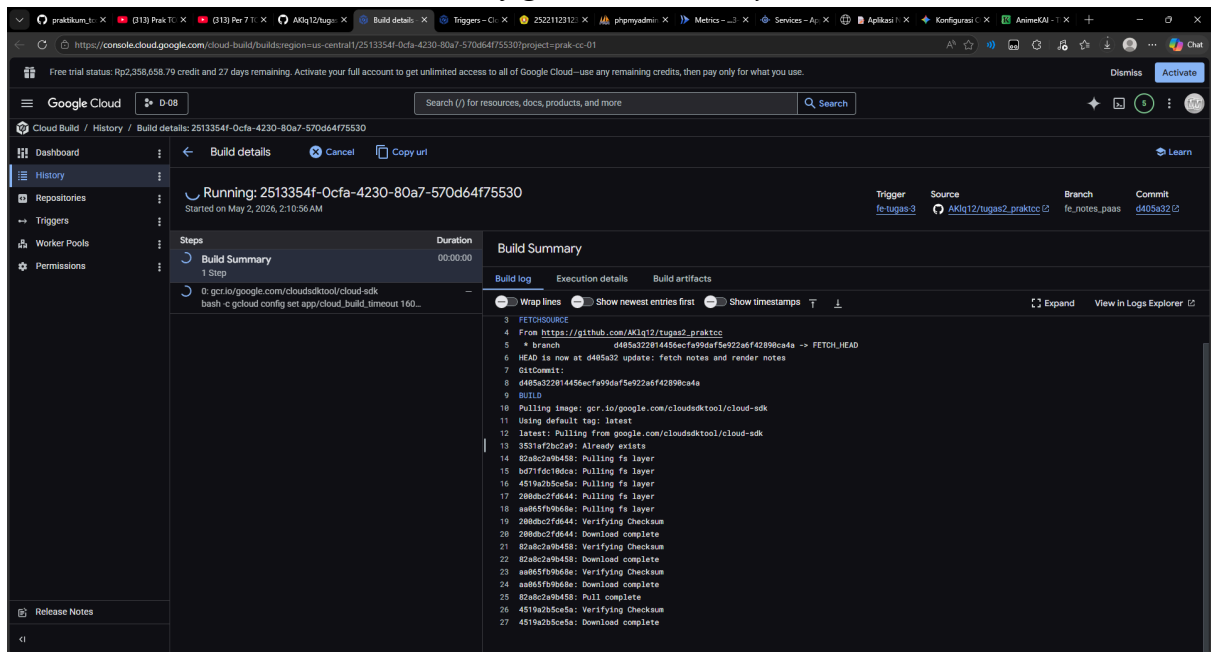
Gambar 5.5 File Konfigurasi cloudbuild.yaml di frontend



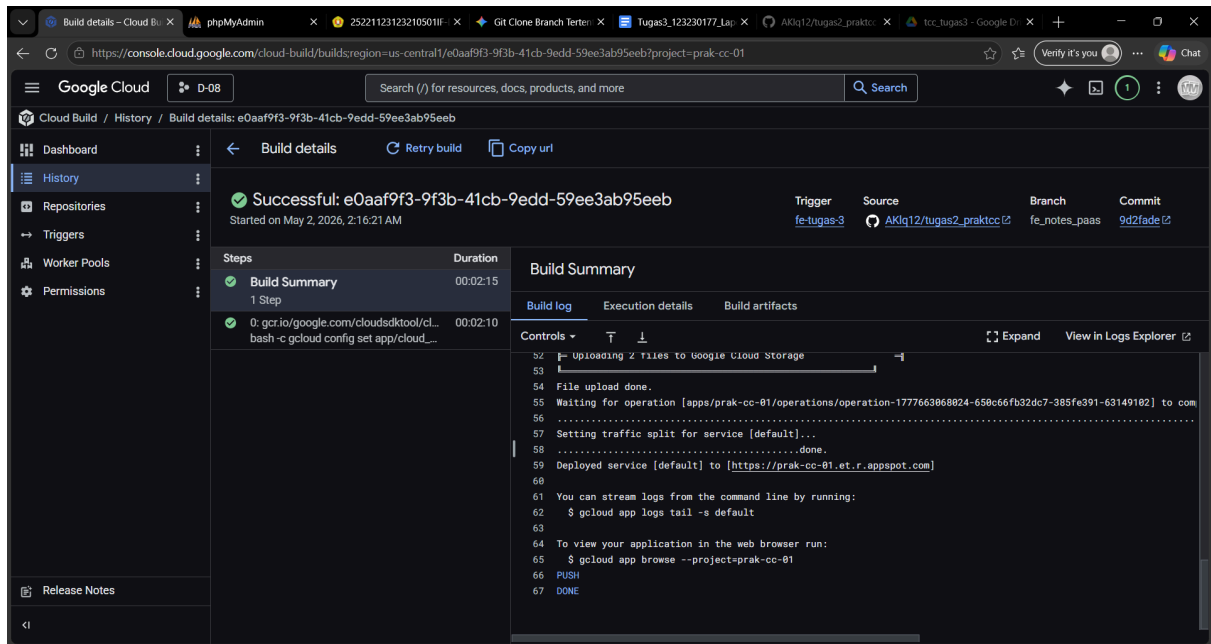
Gambar 5.6 File Konfigurasi Dockerfile di backend



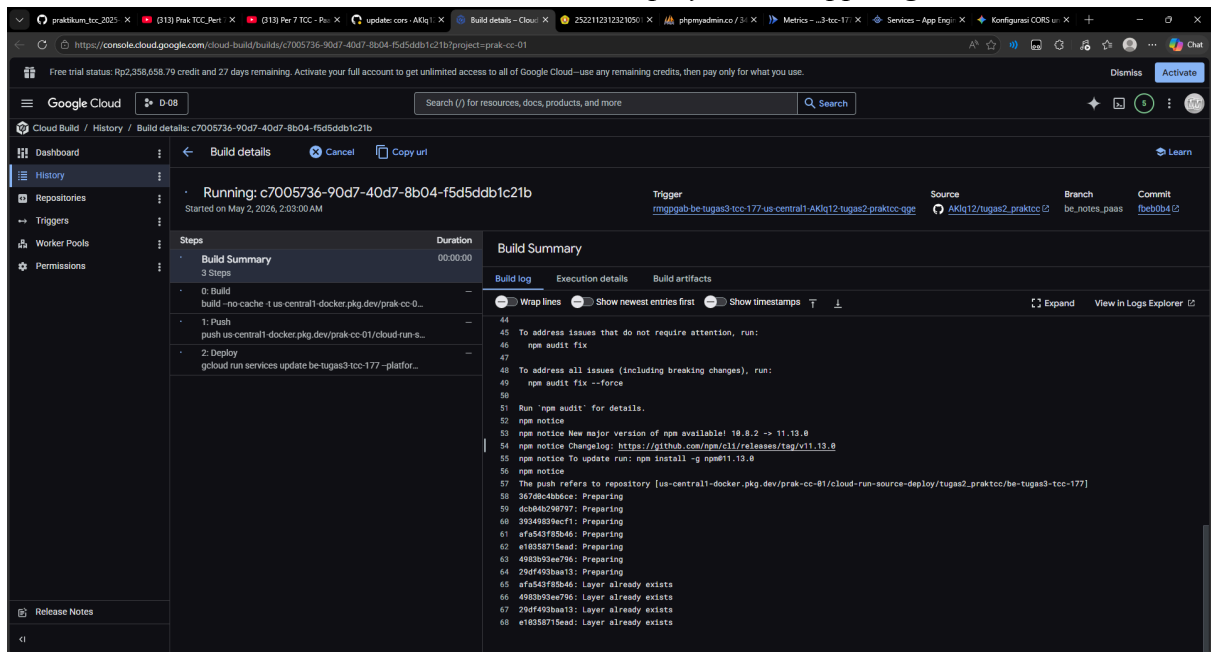
Gambar 5.7 File Konfigurasi cloudbuild.yaml di backend



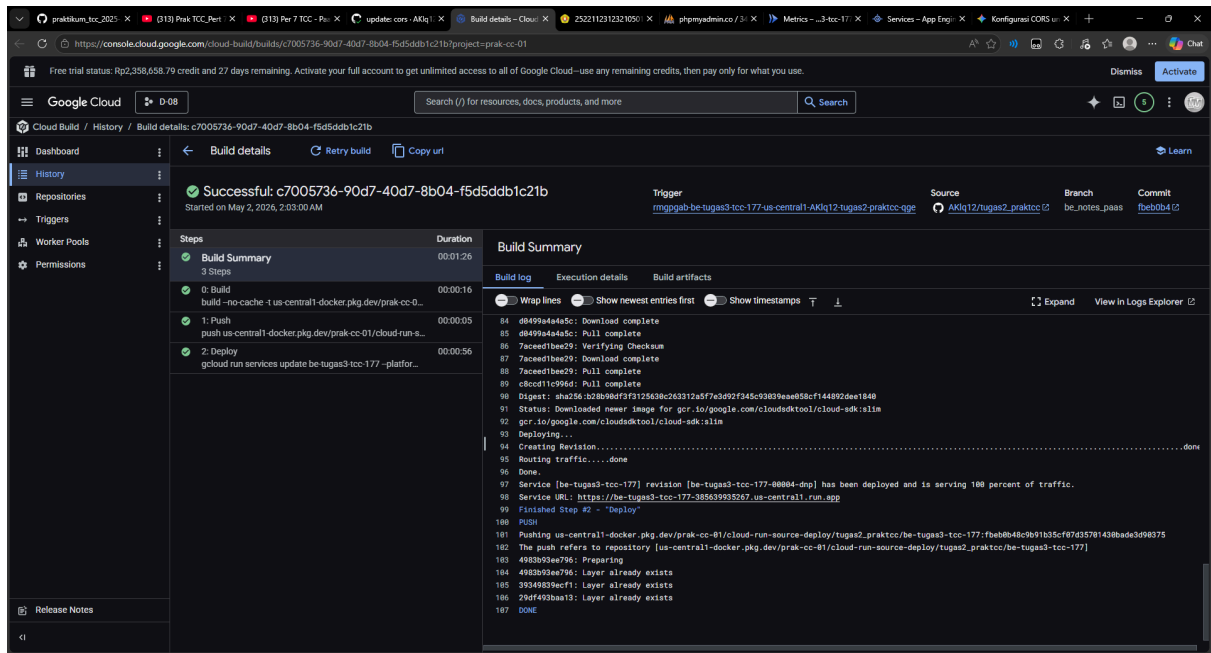
Gambar 5.8 Proses Deployment ke App Engine



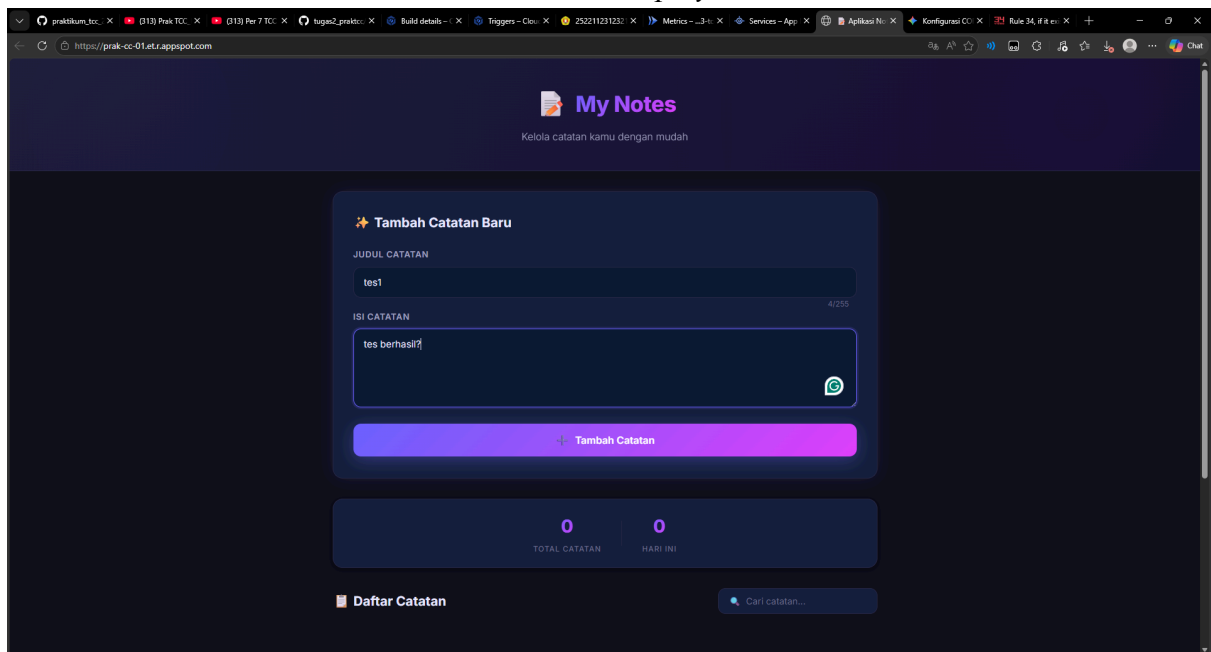
Gambar 5.9 Service berhasil Deployment di App Engine



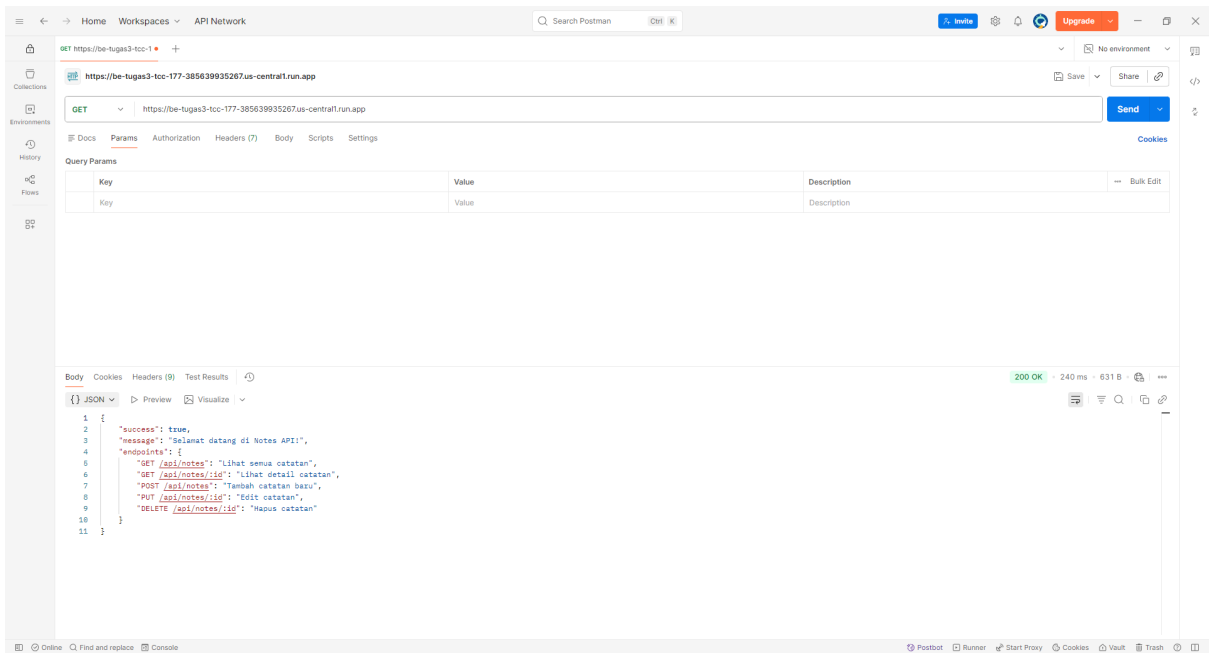
Gambar 6.0 Proses Deployment ke Cloud Run



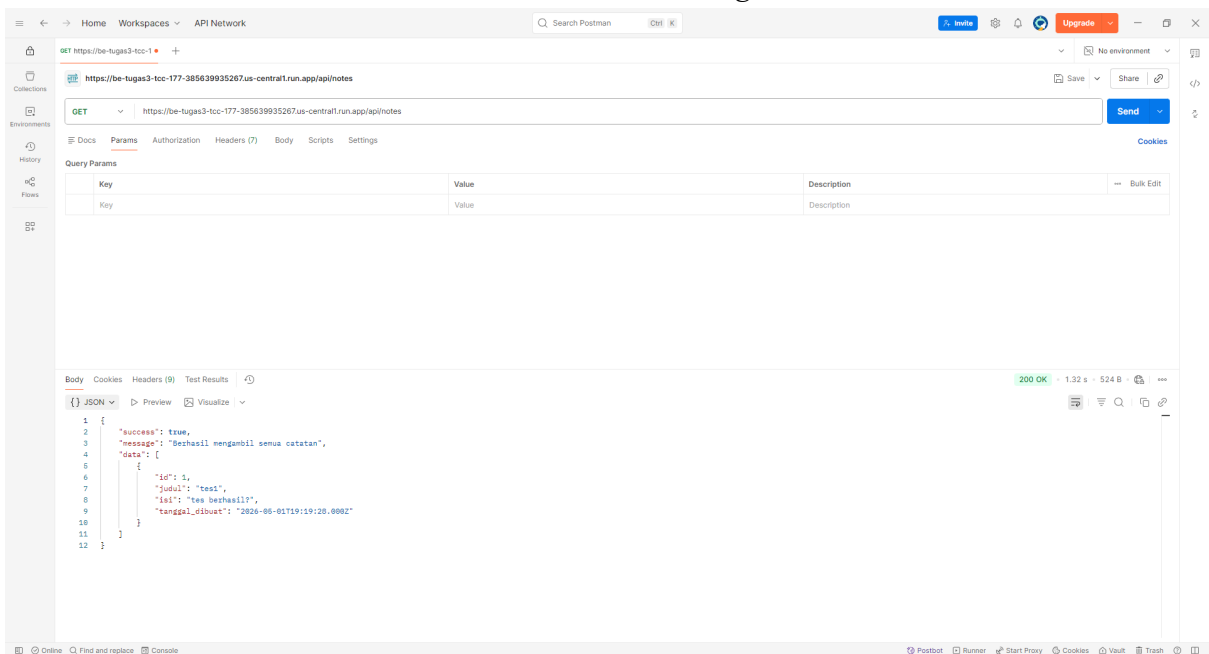
Gambar 6.1 Service berhasil Deployment di Cloud Run



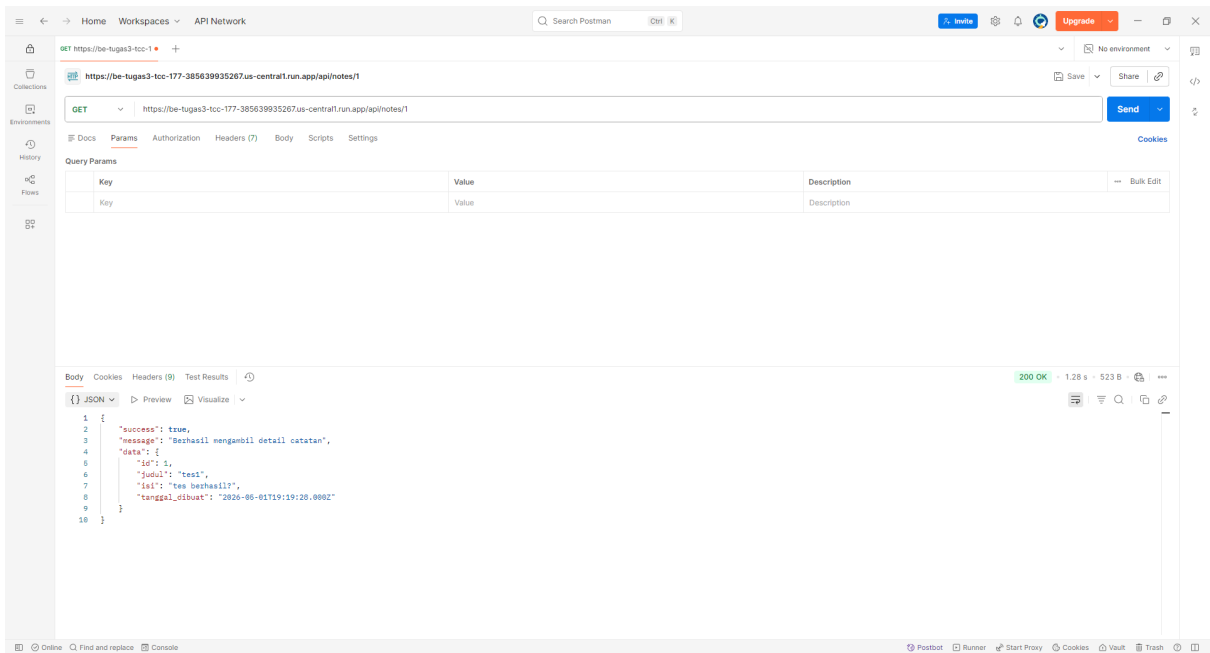
Gambar 6.2 Service Berhasil Diakses Melalui URL



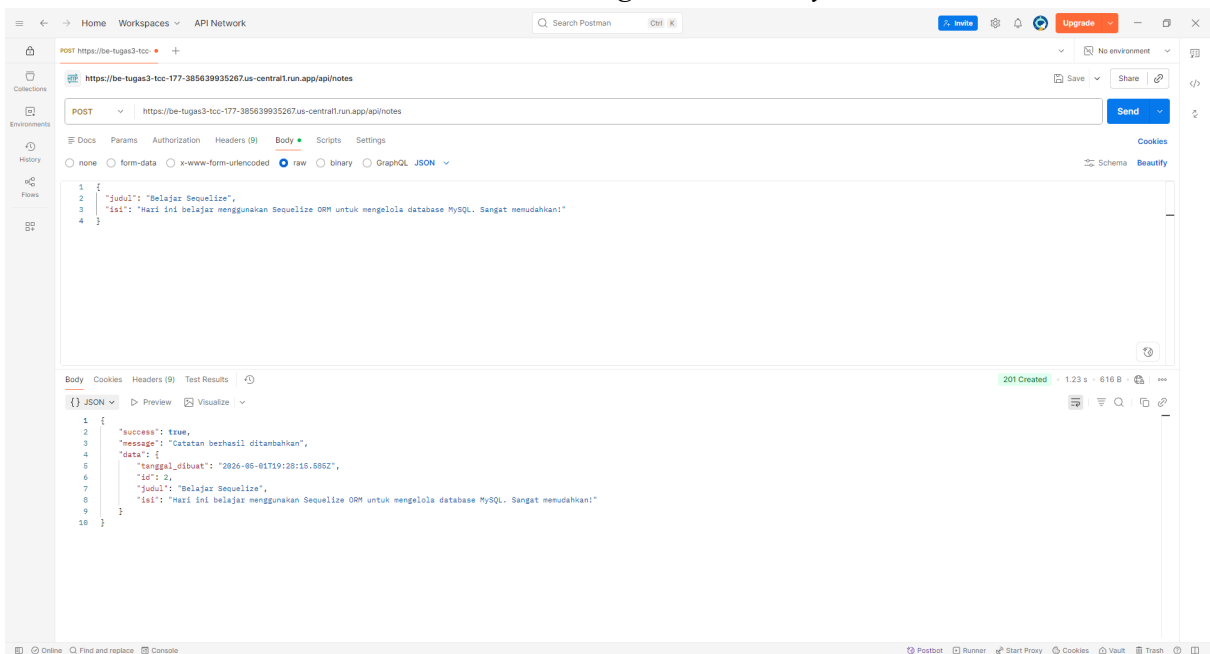
Gambar 6.3 API Testing



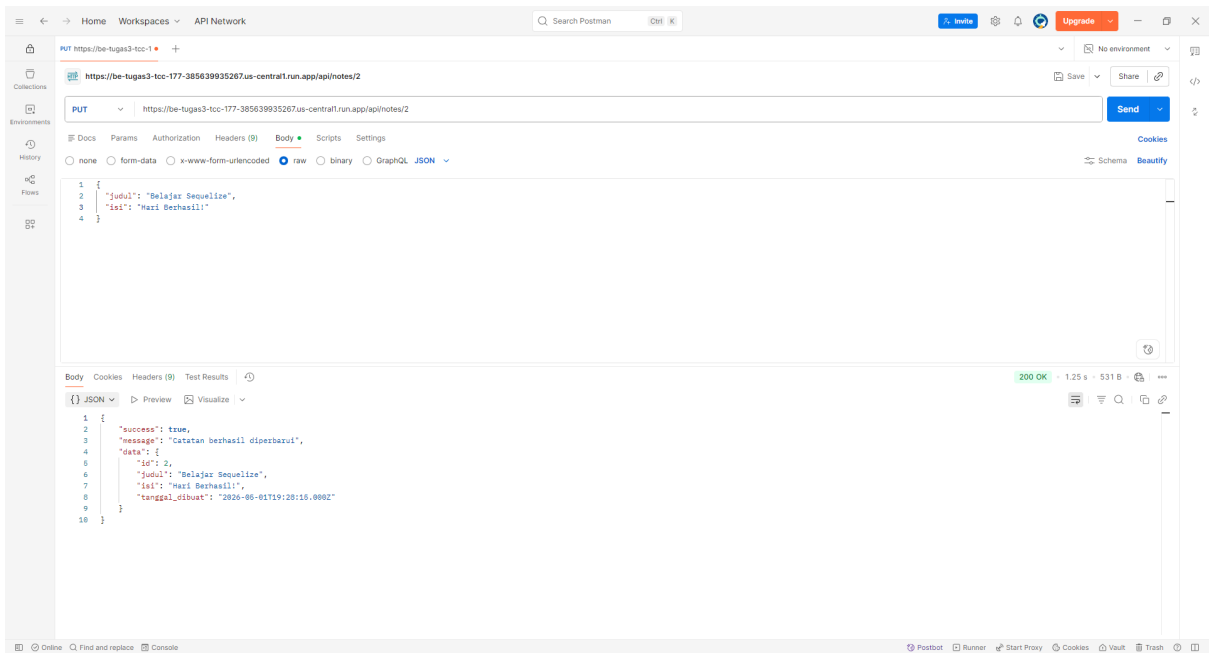
Gambar 6.4 API Testing GET : All Notes



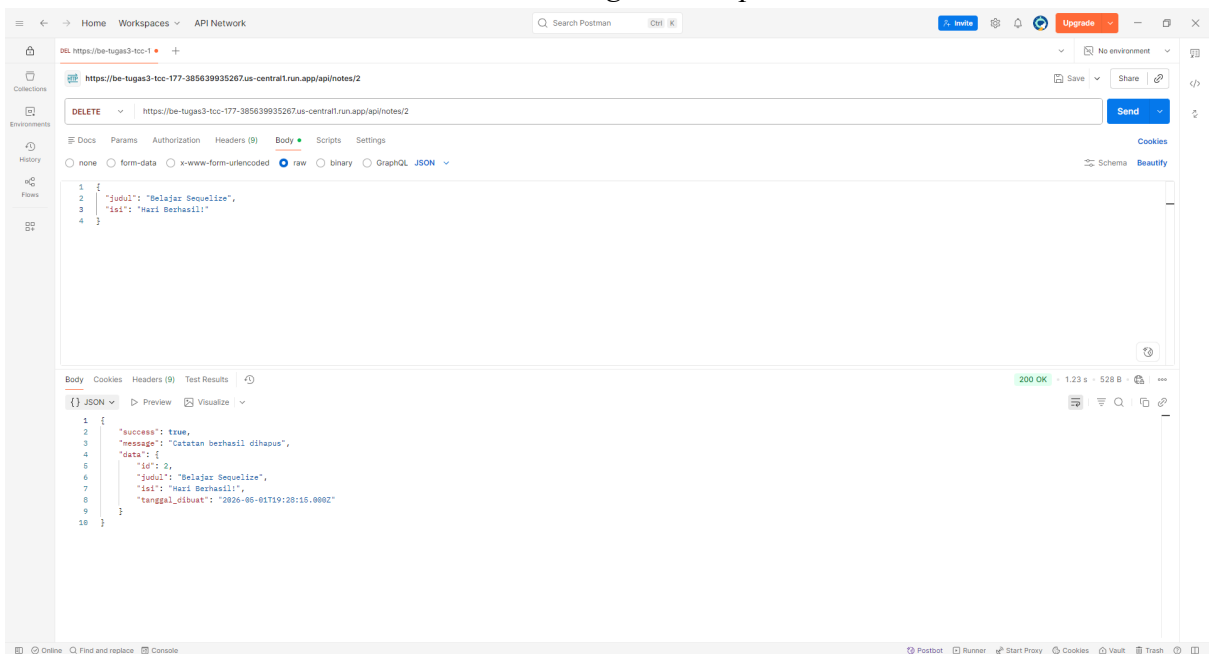
Gambar 6.5 API Testing GET : Note by ID



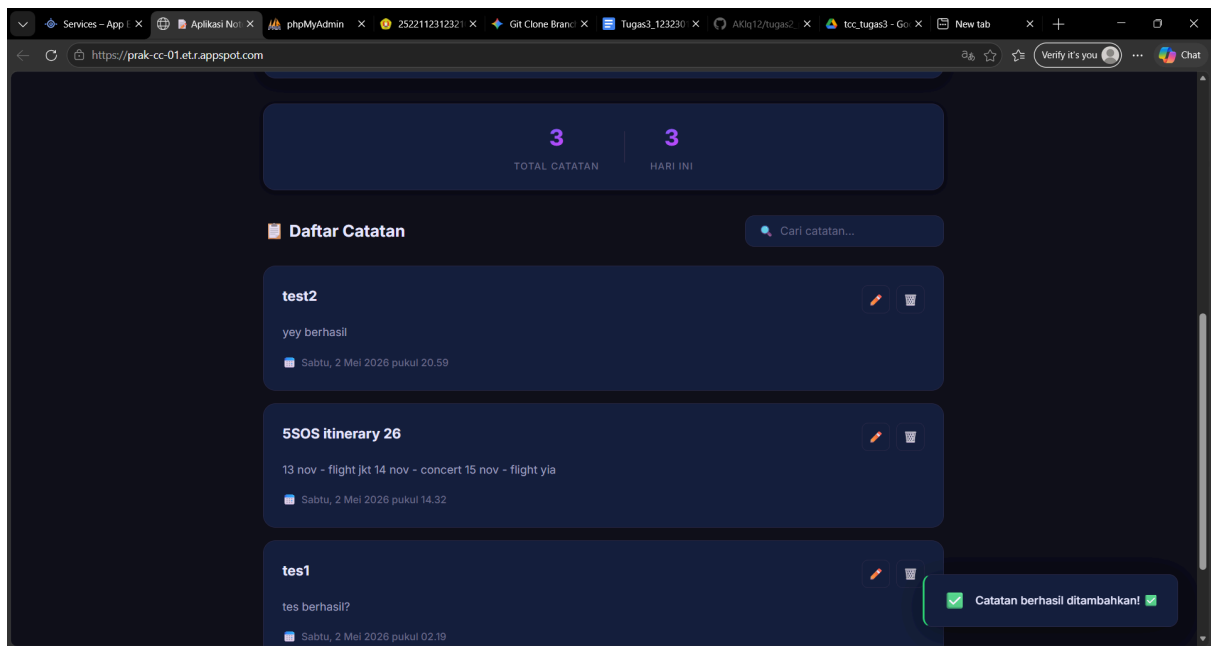
Gambar 6.6 API Testing POST : Create Note



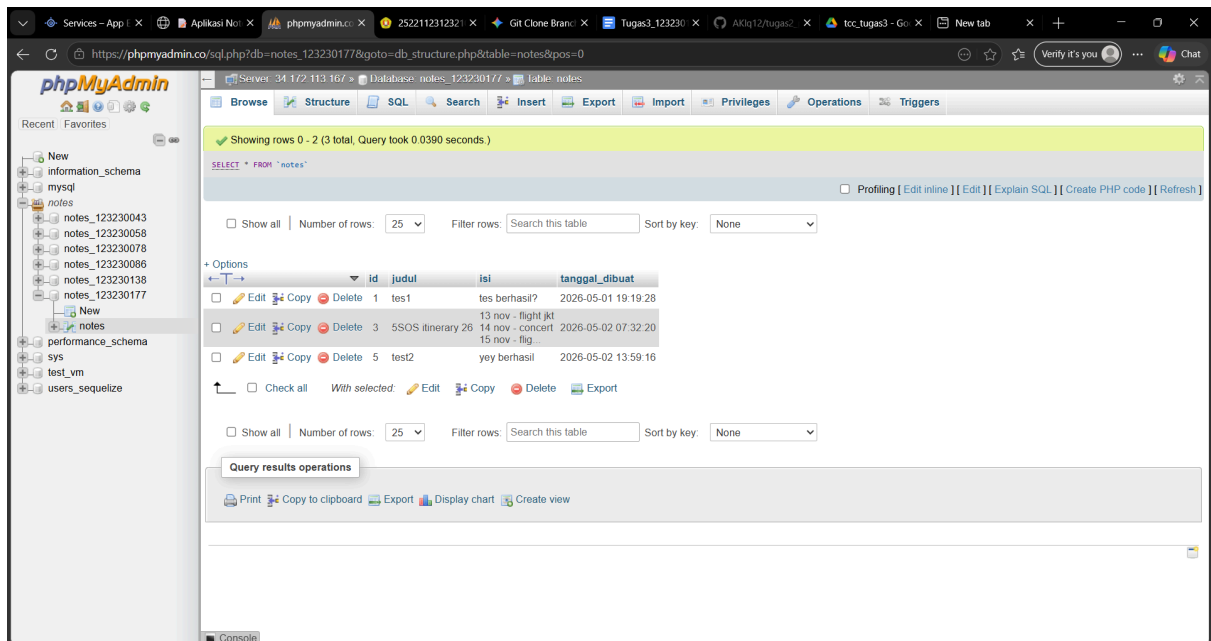
Gambar 6.7 API Testing PUT : Update Note



Gambar 6.8 API Testing DELETE : Delete Note



Gambar 6.9 Frontend berhasil melakukan CRUD melalui Backend.



Gambar 6.9 Frontend berhasil melakukan CRUD melalui Backend